

00 — Setup and Data Preparation (Indian Music Genre)

Downloads the Kaggle dataset `winchester19/indian-music-genre-dataset` via KaggleHub and prepares a leak-free split of mel-spectrogram arrays under `data/processed_indian/`.

```
In [5]: # Logger and config
import os, shutil, json, time, warnings
from pathlib import Path
from datetime import datetime

VERBOSE = os.environ.get('IND_VERBOSE', '1') == '1'
def log(msg: str, level: str='INFO'):
    if not VERBOSE and level=='INFO': return
    print(f'[{datetime.now().strftime('%H:%M:%S')}] {level}: {msg}')

PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
DATA_ROOT = PROJECT_ROOT/'data'
RAW = DATA_ROOT/'indian_music'
PROCESSED = DATA_ROOT/'processed_indian'
PROCESSED.mkdir(parents=True, exist_ok=True)

# Audio/Mel config
SR = int(os.environ.get('SR', '22050'))
N_MELS = int(os.environ.get('N_MELS', '128'))
HOP = int(os.environ.get('HOP', '256'))
SEG_DUR = float(os.environ.get('SEG_DUR', '3.0'))
N_SEGMENTS = int(os.environ.get('N_SEGMENTS', '10'))
T_TARGET = int(os.environ.get('T_TARGET', '128'))

# Parallel/extraction toggles
PARALLEL = os.environ.get('IND_PARALLEL', '1')== '1'
N_JOBS = int(os.environ.get('IND_JOBS', '8'))
MAX_TRACKS_PER_CLASS = int(os.environ.get('IND_MAX_PER_CLASS', '0')) # 0 = r

CONFIG = dict(SR=SR, N_MELS=N_MELS, HOP=HOP, SEG_DUR=SEG_DUR, N_SEGMENTS=N_S
```

```
In [6]: # Download via KaggleHub (idempotent copy into data/indian_music)
import kagglehub
RAW.mkdir(parents=True, exist_ok=True)
DATASET_ID = os.environ.get('INDIAN_DATASET_ID', 'winchester19/indian-music-
log(f'Downloading dataset: {DATASET_ID}')
cache_path = kagglehub.dataset_download(DATASET_ID)
log(f'KaggleHub cache path: {cache_path}')
import pathlib
copied, skipped = 0, 0
for item in pathlib.Path(cache_path).iterdir():
    if not item.is_dir(): continue
    dest = RAW/item.name
```

```

        if dest.exists(): skipped += 1; continue
        shutil.copytree(item, dest); copied += 1
    log(f'Prepared {RAW} - copied={copied}, skipped={skipped}')
    print('Subfolders:', sorted([p.name for p in RAW.iterdir() if p.is_dir()]))

```

```

[18:30:43] INFO: Downloading dataset: winchester19/indian-music-genre-dataset
[18:30:43] INFO: KaggleHub cache path: /home/alepot55/.cache/kagglehub/datasets/winchester19/indian-music-genre-dataset/versions/1
[18:30:43] INFO: Prepared /home/alepot55/Desktop/projects/naml_project/data/indian_music - copied=0, skipped=1
Subfolders: ['genrenew']

```

```

In [9]: # Build file list and labels from subfolders with nested-structure handling
from pathlib import Path
import numpy as np, librosa
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Detect immediate subfolders at RAW level
classes_top = sorted([p.name for p in RAW.iterdir() if p.is_dir()])

# Collect files recursively and derive labels robustly
files, labels = [], []
allowed_ext = {'.mp3', '.wav', '.m4a', '.flac'}
for fp in RAW.rglob('*'):
    if not fp.is_file() or fp.suffix.lower() not in allowed_ext:
        continue
    rel_parts = fp.relative_to(RAW).parts
    if len(classes_top) > 1:
        # Use the first segment under RAW as class
        lbl = rel_parts[0] if len(rel_parts) >= 1 else fp.parent.name
    else:
        # Single top-level folder -> take the next segment as class if available
        lbl = rel_parts[1] if len(rel_parts) >= 2 else fp.parent.name
    files.append(str(fp))
    labels.append(lbl)

# Compute discovered classes from labels
classes = sorted(set(labels))
log(f"Discovered classes: {classes}")

# Optional per-class cap for quick runs
if MAX_TRACKS_PER_CLASS > 0:
    from collections import defaultdict; import random
    byc = defaultdict(list)
    for f, y in zip(files, labels): byc[y].append(f)
    files, labels = [], []
    rng = random.Random(42)
    for y, lst in byc.items(): lst = sorted(lst); rng.shuffle(lst); take = ls

# Encode labels
y_text = labels
le = LabelEncoder().fit(sorted(set(y_text)))
y = le.transform(y_text)

```

```

# Stratify guards
from collections import Counter
cnts = Counter(y)
min_count = min(cnts.values()) if cnts else 0
strat = y if len(set(y))>1 and min_count>=2 else None
Xtr_f, Xte_f, ytr, yte = train_test_split(files, y, test_size=0.2, random_st
strat_tv = ytr if strat is not None else None
Xtr_f, Xva_f, ytr, yva = train_test_split(Xtr_f, ytr, test_size=0.25, random
log(f'Train/Val/Test files: {len(Xtr_f)}/{len(Xva_f)}/{len(Xte_f)} | classes

```

[18:44:47] INFO: Discovered classes: ['bollypop', 'carnatic', 'ghazal', 'sem
iclassical', 'sufi']

[18:44:47] INFO: Train/Val/Test files: 300/100/100 | classes=5

```

In [10]: # Feature extraction (mel-spectrograms) with padding/cropping to T_TARGET ar
import numpy as np, librosa, subprocess
from tqdm import tqdm

def load_audio_librosa(fp: str, sr: int, duration: float):
    """Decode audio using librosa (soundfile/audioread)."""
    y, _ = librosa.load(fp, sr=sr, mono=True, duration=duration, res_type='k
    return y

def load_audio_ffmpeg(fp: str, sr: int, duration: float):
    """Decode audio via ffmpeg pipe to float32 PCM; returns np.ndarray or No

    Requires ffmpeg to be installed and available on PATH.
    """
    cmd = [
        "ffmpeg", "-v", "error", "-i", fp,
        "-ac", "1", "-ar", str(sr)
    ]
    if duration and duration > 0:
        cmd += ["-t", str(duration)]
    cmd += ["-f", "f32le", "pipe:1"]
    try:
        proc = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess
        audio = np.frombuffer(proc.stdout, dtype=np.float32)
        return audio
    except Exception:
        return None

def load_audio(fp: str, sr: int, duration: float):
    """Try librosa first, then ffmpeg fallback; raise if both fail."""
    try:
        y = load_audio_librosa(fp, sr, duration)
        if y is not None and y.size > 0:
            return y
    except Exception:
        pass
    y = load_audio_ffmpeg(fp, sr, duration)
    if y is None or y.size == 0:
        raise RuntimeError(f"Failed to decode audio: {fp}")

```

```

    return y

def extract_one(fp: str):
    try:
        y = load_audio(fp, SR, SEG_DUR * N_SEGMENTS)
        seg_len = int(SR * SEG_DUR)
        out = []
        for s in range(N_SEGMENTS):
            st, en = s * seg_len, s * seg_len + seg_len
            if st >= len(y):
                break
            seg = y[st:en]
            if len(seg) < seg_len:
                seg = np.pad(seg, (0, seg_len - len(seg)))
            mel = librosa.feature.melspectrogram(y=seg, sr=SR, n_mels=N_MELS)
            mel_db = librosa.power_to_db(mel, ref=np.max)
            T = mel_db.shape[1]
            if T > T_TARGET:
                mel_db = mel_db[:, :T_TARGET]
            elif T < T_TARGET:
                mel_db = np.pad(mel_db, ((0, 0), (0, T_TARGET - T)))
            out.append(mel_db)
        return out
    except Exception:
        return []

def extract_batch(file_list, y_list):
    X_list, y_seg = [], []
    for fp, y_lbl in tqdm(list(zip(file_list, y_list)), total=len(file_list)):
        segs = extract_one(fp)
        if len(segs) == 0:
            continue
        X_list.extend(segs)
        y_seg.extend([y_lbl] * len(segs))
    return X_list, np.asarray(y_seg, dtype=np.int64)

Xtr_list, ytr_seg = extract_batch(Xtr_f, ytr)
Xva_list, yva_seg = extract_batch(Xva_f, yva)
Xte_list, yte_seg = extract_batch(Xte_f, yte)
log(f"Extracted segments -> train={len(Xtr_list)}, val={len(Xva_list)}, test={len(Xte_list)}")

def to_array(lst):
    if len(lst) == 0:
        return np.empty((0, N_MELS, T_TARGET, 1), dtype=np.float32)
    return np.asarray(lst, dtype=np.float32)[..., None]

Xtr, Xva, Xte = to_array(Xtr_list), to_array(Xva_list), to_array(Xte_list)

# Fit scaler on train only
from sklearn.preprocessing import StandardScaler

```

```

scaler = StandardScaler()

def scale_3d(X, fit=False):
    if X.size == 0:
        return X
    sh = X.shape
    Z = X.reshape(sh[0], -1)
    Z = scaler.fit_transform(Z) if fit else scaler.transform(Z)
    return Z.reshape(sh).astype(np.float32)

if Xtr.shape[0] == 0:
    n_files = len(Xtr_f)
    log(f"No training segments extracted from {n_files} training files.", le
    log("Hints: ensure ffmpeg is installed and accessible, and RAW contains
    raise ValueError("Training set is empty after feature extraction.")

Xtr = scale_3d(Xtr, fit=True)
Xva = scale_3d(Xva)
Xte = scale_3d(Xte)

# Persist arrays and transformers
np.save(PROCESSED / 'X_train.npy', Xtr)
np.save(PROCESSED / 'y_train.npy', ytr_seg)
np.save(PROCESSED / 'X_val.npy', Xva)
np.save(PROCESSED / 'y_val.npy', yva_seg)
np.save(PROCESSED / 'X_test.npy', Xte)
np.save(PROCESSED / 'y_test.npy', yte_seg)
import pickle
with open(PROCESSED / 'label_encoder.pkl', 'wb') as f:
    pickle.dump(le, f)
with open(PROCESSED / 'scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
log(f'Saved processed arrays to {PROCESSED}')

```

```

0%|          | 0/300 [00:00<?, ?it/s]
100%|██████████| 300/300 [03:59<00:00, 1.25it/s]
100%|██████████| 300/300 [03:59<00:00, 1.25it/s]
100%|██████████| 100/100 [01:18<00:00, 1.27it/s]
100%|██████████| 100/100 [01:18<00:00, 1.27it/s]
100%|██████████| 100/100 [01:16<00:00, 1.31it/s]

```

[18:51:23] INFO: Extracted segments -> train=3000, val=1000, test=1000

[18:51:23] INFO: Saved processed arrays to /home/alepot55/Desktop/projects/naml_project/data/processed_indian